

Q Shape Test - Python

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.fftpack import fft2, fftshift

# Simulation parameters
size = 50 # Grid size
steps = 5000 # Longer evolution time
diffusion_rates = [0.1, 0.5, 1.0] # Different diffusion strengths
perturbation_strengths = [0.01, 0.1, 0.5] # Increasing perturbations

# Function to create different initial geometries
def create_geometry(shape, size):
    grid = np.zeros((size, size))

    if shape == "nested_rings":
        for r in range(5, size//2, 10):
            y, x = np.ogrid[-size//2:size//2, -size//2:size//2]
            mask = (x**2 + y**2 >= r**2) & (x**2 + y**2 < (r+3)**2)
            grid[mask] = 1

    elif shape == "double_toroidal":
        for r in [10, 20]:
            y, x = np.ogrid[-size//2:size//2, -size//2:size//2]
            mask = (x**2 + y**2 >= r**2) & (x**2 + y**2 < (r+3)**2)
            grid[mask] = 1

    elif shape == "spiral":
        theta = np.linspace(0, 4*np.pi, 500)
        r = np.linspace(5, size//2 - 1, 500) # Prevent index overflow
        x = np.clip((size//2 + r * np.cos(theta)).astype(int), 0, size-1)
        y = np.clip((size//2 + r * np.sin(theta)).astype(int), 0, size-1)
        grid[x, y] = 1

    elif shape == "hexagonal":
        for i in range(0, size, 5):
            for j in range(0, size, 5):
                if (i + j) % 10 == 0:
                    grid[i, j] = 1

    elif shape == "fractal_wave":
```

```

x = np.arange(size)
y = np.arange(size)
X, Y = np.meshgrid(x, y)
grid = np.sin(2 * np.pi * X / 10) * np.sin(2 * np.pi * Y / 10)
grid = (grid - np.min(grid)) / (np.max(grid) - np.min(grid)) # Normalize

return grid

# Evolution function with overflow prevention
def evolve(grid, diffusion, perturbation, steps):
    evolved = grid.copy()

    for _ in range(steps):
        laplacian = (
            np.roll(evolved, 1, axis=0) + np.roll(evolved, -1, axis=0) +
            np.roll(evolved, 1, axis=1) + np.roll(evolved, -1, axis=1) -
            4 * evolved
        )
        evolved += diffusion * laplacian + perturbation * np.random.randn(*evolved.shape)
        evolved = np.clip(evolved, -1, 1) # Prevent extreme values

    return evolved

# Visualization function
def plot_results(initial, evolved, shape, diffusion, perturbation):
    fig, axes = plt.subplots(2, 3, figsize=(12, 8))

    # Initial condition
    axes[0, 0].imshow(initial, cmap="hot")
    axes[0, 0].set_title(f"Initial {shape}")

    # Evolved state
    axes[0, 1].imshow(evolved, cmap="hot")
    axes[0, 1].set_title(f"Evolved {shape} (D={diffusion}, P={perturbation})")

    # Difference map
    difference = np.abs(initial - evolved)
    axes[0, 2].imshow(difference, cmap="hot")
    axes[0, 2].set_title(f"Difference Map ({shape})")

    # Fourier transforms
    ft_initial = fftshift(np.abs(fft2(initial)))
    ft_evolved = fftshift(np.abs(fft2(evolved)))

```

```

axes[1, 0].imshow(np.log(1 + ft_initial), cmap="magma")
axes[1, 0].set_title(f"Fourier Initial ({shape})")

axes[1, 1].imshow(np.log(1 + ft_evolved), cmap="magma")
axes[1, 1].set_title(f"Fourier Evolved ({shape})")

ft_diff = np.abs(ft_initial - ft_evolved)
axes[1, 2].imshow(np.log(1 + ft_diff), cmap="magma")
axes[1, 2].set_title(f"Fourier Difference ({shape})")

plt.tight_layout()
plt.show()

# Run experiments for all geometries
geometries = ["nested_rings", "double_toroidal", "spiral", "hexagonal", "fractal_wave"]

for shape in geometries:
    for diffusion in diffusion_rates:
        for perturbation in perturbation_strengths:
            initial = create_geometry(shape, size)
            evolved = evolve(initial, diffusion, perturbation, steps)
            plot_results(initial, evolved, shape, diffusion, perturbation)

```